

Laboratorio 4 - Sviluppo di un semplice viewer 3D basato su camera orbitante

In questa esercitazione svilupperemo un visualizzatore di oggetti 3D basato sul paradigma della camera orbitante:

1. La scena viene modellata direttamente in coordinate normalizzate, ossia, per essere visibili i vertici devono avere tutte le coordinate in $[-1,1]$.
2. Nel codice fornito, la scena è modellata nella classe `Scene` ed è costituita di soli due triangoli che hanno un vertice in comune e sono messi “a tetto”, come due facce inclinate opposte di una piramide.
3. Scopo del laboratorio è visualizzare tale scena applicando la trasformazione di vista, che dipende dalla posizione della camera orbitante, e la trasformazione di proiezione, che dipende dai parametri intrinseci della camera.
4. I parametri di vista, sia intrinseci che estrinseci, dovranno essere controllabili tramite il movimento del mouse.

Procediamo in quattro tappe a partire dal template `00-butterfly.cc`. Il codice iniziale è quasi identico a quello sviluppato nel laboratorio precedente, ad eccezione della scena rappresentata, che è costituita da due triangoli, come spiegato sopra.

Capitolo 1 - costruzione del vertex shader che gestisce le trasformazioni di vista e di proiezione

In questa tappa non modifichiamo il codice C++, ma costruiamo un vertex shader in grado di gestire la trasformazione tramite la camera orbitante. Per ora, tutti i parametri della camera orbitante e di proiezione sono costanti nello shader. Consultare prima di tutto le slide di teoria per vedere che matrici servono per la camera orbitante: rotazioni attorno agli assi x e y e traslazione lungo l'asse z .

1. Per prima cosa, è necessario assemblare le matrici di rotazione attorno agli assi y e x , che dipendono rispettivamente dai seni e coseni degli angoli `theta_deg` e `phi_deg`, entrambi espressi in gradi: calcolare i seni e coseni (ricordandosi di riscaldare gli angoli in radianti) e assemblare le matrici in due variabili `mat4`, ciascuna delle quali viene inizializzata elencando i 16 coefficienti di seguito. **Attenzione:** le matrici glsl sono codificate in ordine column-major, quindi i coefficienti vanno elencati scandendo la matrice per colonne (non per righe come verrebbe naturale).
2. Quindi assembliamo in modo analogo la matrice di traslazione lungo l'asse z : poniamo il centro della scena a $z=-2.0$.
- Infine, assembliamo la matrice di proiezione. Scegliamo una distanza focale `fd=2.0`, poniamo le distanze dei piani di clip a `fcp=3.0` e `ncp=1.0` e assembliamo la matrice secondo la formula:

$$pr = \begin{bmatrix} fd & 0 & 0 & 0 \\ 0 & fd & 0 & 0 \\ 0 & 0 & -\frac{f_{cp}+n_{cp}}{f_{cp}-n_{cp}} & -\frac{2f_{cp}n_{cp}}{f_{cp}-n_{cp}} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

1. Questa formula è più semplice rispetto a quella generale vista a lezione, perché stiamo considerando una scena definita già all'interno del volume canonico $[0,1] \times [0,1] \times [0,1]$.
2. Assembliamo la matrice composta moltiplicando le matrici di rotazione, la matrice di traslazione e quella di proiezione **nell'ordine giusto!**
3. Modifichiamo la posizione del vertice in input mediante la matrice composta così ottenuta e assegniamo tale valore al parametro di output `gl_Position`.
4. Sperimentiamo lo shader così ottenuto cambiando manualmente i valori delle costanti `phi_deg` e `theta_deg` e facendo hot load dello shader senza far riparire il programma. Si devono vedere i due triangoli che compongono l'oggetto da diversi punti di vista, secondo i valori assegnati. Utilizzare valori in $[-180,180]$ per `phi_deg` e in $[-90,90]$ per `theta_deg`.

Capitolo 2 - gestione della rotazione tramite il mouse

Aggiungiamo ora il controllo della rotazione tramite mouse. Vogliamo che i parametri della rotazione possano cambiare tramite trascinamento (drag) del puntatore, e che questo avvenga solo quando l'utente tiene premuto il bottone sinistro del mouse. In particolare, vogliamo che la componente orizzontale del trascinamento influenzi l'angolo `phi`, e quella verticale l'angolo `theta`. Per sviluppare questo punto, è utile rivedere la parte di teoria sulla camera orbitante.

1. Cambiamo prima di tutto i due simboli nel fragment shader, trasformandoli da `const` a `uniform`. Adeguiamo il programma C++ di conseguenza, creando una nuova classe `Camera` per gestire la camera orbitante, contenente:
 - due membri `phi_deg` e `theta_deg`, copie lato CPU dei valori corrispondenti;
 - due membri con le location necessarie ad agganciare le uniform corrispondenti in GPU;
 - una funzione membro privata `update` per aggiornare il valore delle uniform in GPU;
 - una funzione membro pubblica `drag` che prende in input due valori `float dx` e `float dy` (offset del mouse in pixel) e li usi per aggiornare i valori di `phi_deg` e `theta_deg`, sia in CPU che in GPU;
 - nella funzione `drag`, un fattore di scala dei valori `dx` e `dy`, per regolare la velocità di risposta della camera; quale può essere un buon fattore di scala iniziale?
 - nella funzione `drag`, limitazione del valore di `theta_deg` al range $[-90,90]$.
 - un costruttore esplicito che prenda in input lo shader program, agganci le variabili location alle uniform corrispondenti e le inizializzi.

2. Inizializzare una camera nel main. A questo punto, il programma deve girare mostrando la scena nell'orientamento corrispondente ai valori iniziali delle uniform; non è più possibile cambiare l'orientamento con hot reload dello shader; per farlo, è necessario cambiare i valori di default e ricompilare il programma.
3. Aggiungiamo una funzione callback (overload di `handle`) per gestire il movimento del mouse. Questa funzione:
 - prende in input il mouse e la camera;
 - calcola *sempre* gli offset delle coordinate del mouse rispetto al frame precedente;
 - *solo se il bottone sinistro è premuto*, usa l'offset calcolato per aggiornare la camera mediante la funzione membro `drag` spiegata al punto precedente;
 - ha due variabili `static float` che ricordano la posizione del mouse al frame precedente, necessarie per il calcolo degli offset relativi del mouse di frame in frame, per ora inizializzate a 0 (ricordate: l'inizializzazione di una variabile `static` avviene solo al lancio del programma, in tutte le invocazioni successive della funzione mantiene l'ultimo valore calcolato);
 - infine, invoca `camera.drag()` usando come parametri le coordinate di spostamento relativo del mouse.
4. Aggiungiamo la gestione della nuova `handle` nel main loop. A questo punto, il programma deve consentire l'orientamento interattivo della camera attraverso il mouse: facendo drag del mouse nella finestra, si vede girare l'oggetto. Sperimentare con diverse velocità (fattori di scala dell'update) e diverse direzioni di movimento (segno dell'update, per simulare la manipolazione dell'oggetto o della camera).

Capitolo 3 - Spostamento dei parametri di vista nel programma C++

A questo punto, osserviamo che fare tutti i calcoli delle matrici di trasformazione nello shader non è il modo più efficiente: il calcolo è lo stesso per tutti i vertici della scena, ma viene ripetuto per ciascuno di essi (e, in generale, potrebbero essere milioni). Conviene quindi spostare questo calcolo in CPU, effettuarlo una sola volta e passare la matrice `vp` risultante come uniform al vertex shader. Per farlo, includiamo nel programma la [libreria glm](#), che fornisce i tipi e gli operatori dell'algebra lineare (vettori, matrici e operazioni dell'algebra) analoghi a quelli di glsl.

1. Conviene inserire la dipendenza a glm come libreria scaricata direttamente da cmake, e inclusa in modalità header only.
 - aggiungere in `CMakeLists.txt`, dopo il fetch di SFML:

```
FetchContent_Declare(
    glm
```

```

GIT_REPOSITORY https://github.com/g-truc/glm.git
GIT_TAG        1.0.3
)
FetchContent_MakeAvailable(glm)
    • nel programma C++ non è necessario includere tutta glm, ma solo le
      operazioni relative alle matrixi 4x4 e alla trigonometria:
#include <glm/mat4x4.hpp>
#include <glm/trigonometric.hpp>.
2. Inseriamo nel vertex shader una nuova uniform mat4 vp e rimuoviamo
   le uniform precedenti e tutto il codice che calcola la matrice di vista e
   proiezione: lo shader si riduce a moltiplicare la posizione del punto per la
   matrice vp e restituire il risultato nel parametro gl_Position. Salviamo
   il codice rimosso in un file, per usarlo dopo.
3. Modifichiamo la classe Camera eliminando la gestione delle uniform prece-
   denti (non più presenti) e sostituendola con la gestione della uniform vp:
   le variabili locali alla classe per mantenere gli angoli di rotazione restano,
   ma le corrispondenti uniform non ci sono più. A parte poche istruzioni per
   consentire l'aggiornamento della uniform vp in GPU, il grosso del codice
   riguarda la funzione membro update: questa deve calcolare esplicitamente
   la vp riproducendo tutti i conti che venivano precedentemente fatti dal
   vertex shader. È sufficiente copiare il codice rimosso dallo shader nel corpo
   della update e, con poche modifiche, trasformarlo in codice C++ che usa
   la libreria glm per la gestione dell'algebra lineare. In particolare, la copia
   locale della vp sarà una variabile della funzione update di tipo glm::mat4,
   che verrà utilizzata per aggiornare il valore della uniform alla fine della
   funzione. A questo punto il programma deve girare con comportamento
   del tutto analogo alla versione precedente.

```

Capitolo 4 - Gestione dello zoom e del dolly

Estendiamo il programma per gestire in modo interattivo le funzioni di zoom e di dolly: lo zoom agisce sulla distanza focale della camera per consentire di inquadrare un campo più largo (grandangolo) o più stretto (teleobiettivo); il dolly permette di allontanare o avvicinare il centro della scena alla camera, agendo sulla traslazione lungo l'asse z che fa parte del calcolo della matrice di vista.

1. Nella versione precedente, questi due parametri sono variabili o costanti **float** locali alla funzione **update** della classe **Camera**. Li trasformiamo in variabili membro della classe e ne gestiamo l'aggiornamento attraverso due nuove funzioni membro **zoom** e **dolly**: entrambe queste funzioni prendono in input un offset di movimento del mouse nella sola direzione y e, analogamente alla funzione **drag**, usano tale offset per modificare rispettivamente il valore del parametro di zoom e di quello di dolly.
2. Modifichiamo la callback **handle** del mouse aggiungendo in fondo l'aggiornamento di zoom o dolly solo se è attivo un dato modificatore,

specifico di quella funzionalità: scegliamo un modificatore per lo zoom e uno diverso per il dolly (ad esempio, i tasti shift, ctrl, alt). A questo punto, il programma deve mantenere le funzionalità precedenti e integrarle con la gestione di zoom e dolly. Notiamo che attivando lo zoom la scena può eccedere i limiti della finestra, ma resta sempre all'interno dei piani di clipping front e back; diversamente, attivando il dolly la scena può eccedere i piani di clip, risultando troncata o sparendo del tutto.

Estensioni ed esercizi aggiuntivi

Come esercizio ulteriore, si possono sviluppare i seguenti aggiustamenti e migliorie:

1. Problema: cosa succede se, alla partenza del programma, uno dei modificatori tastiera inseriti nell'ultimo capitolo è già premuto mentre il mouse è fuori dalla finestra e rientra nella finestra? Anche a programma partito, cosa succede facendo la stessa cosa e facendo rientrare il mouse da un lato diverso da quello da cui era rientrato? Cosa sta succedendo? Suggerimento: esistono due eventi SFML, che possono essere gestiti con specifiche callback, chiamati `MouseEntered` e `MouseLeft`, il cui funzionamento è descritto nella [pagina events](#) dei tutorial.
2. Aggiungere, analogamente all'ultima tappa, la gestione della funzione pan: è sufficiente usare l'offset del mouse, se un ulteriore modificatore o bottone è attivo, per impostare le traslazioni lungo gli assi x e y che vanno integrate nella matrice di traslazione che si usa per il calcolo della vp.
3. Aggiungere la gestione di un tasto di reset, che porta la vista alla posizione iniziale.
4. Aggiungere altri triangoli alla scena, costruendo un oggetto più complesso (senza eccedere, per il momento, dal cubo $[-1,1] \times [-1,1] \times [-1,1]$). Ad esempio, chiudere la farfalla creando una piramide (altre due triangoli a partire dalla cuspide, e altri due triangoli come base) e abilitare nuovamente il culling.

Queste funzionalità non sono sviluppate nella soluzione, ma sono semplici da implementare nel solco di quanto già proposto in questa traccia.